

Sparse Matrix–Vector Multiplication in CSR: Sequential Baseline and OpenMP Parallelization

Alessandro Gremes

Department of Information Engineering and Computer Science (DISI)

University of Trento

Student ID: 242330

Email: alessandro.gremes@studenti.unitn.it

GitHub Repository: <https://github.com/alegrem123/PARCO-Computing-2026-242330>

Abstract—Sparse Matrix–Vector Multiplication (SpMV) is a key kernel in numerical linear algebra and graph analytics. In the CSR format, its low arithmetic intensity and irregular memory access place it in the memory-bandwidth-bound regime predicted by the Roofline model. This work implements a sequential CSR SpMV baseline and an OpenMP version exploiting row-level parallelism. Six real matrices from SuiteSparse are benchmarked on the UniTN HPC cluster (queue `short_cpuQ`) using 2–64 threads and multiple OpenMP scheduling strategies. Each configuration is executed 10 times after warm-up, collecting median and 90th percentile timings. Results show strong matrix-dependent behaviour: large matrices scale efficiently up to 16–32 threads before saturating memory bandwidth, whereas small matrices see no benefit due to parallel overhead. Dynamic and guided scheduling systematically reduce load imbalance on irregular sparsity patterns. Overall, the study confirms that CSR SpMV performance on multicore CPUs is dominated by memory subsystem limits and scheduling-dependent workload distribution.

Index Terms—Sparse Matrix–Vector Multiplication, CSR, OpenMP, scheduling, benchmarking, memory-bound kernels

I. INTRODUCTION

Sparse Matrix–Vector Multiplication (SpMV) is a core kernel in HPC, dominating the cost of Krylov solvers, PDE discretizations, graph analytics, and sparse ML workloads [1]. Its optimization is difficult because sparse formats generate irregular memory access and extremely low arithmetic intensity, making performance primarily bandwidth-limited rather than compute-limited.

The Compressed Sparse Row (CSR) format is the standard on CPUs thanks to its compact, row-contiguous representation. However, indirect indexing of the vector x and highly nonuniform row lengths degrade locality and expose the kernel’s memory-bound nature under the Roofline model [2]. Modern studies and libraries—such as GraphBLAS [5], WISE performance modeling [6], and memory-aware SpMV optimizations [7]—consistently show that bandwidth saturation and load imbalance are the principal scalability bottlenecks for CSR SpMV on multicore systems.

Following PARCO methodology and scientific benchmarking principles [3], this work implements a sequential CSR baseline and an OpenMP row-parallel version. The study evaluates their performance on real SuiteSparse matrices [1],

focusing on speedup, efficiency, scheduling effects, and bandwidth saturation.

II. METHODOLOGY

A. Problem Definition

Given a sparse matrix $A \in \mathbb{R}^{n \times n}$ and a dense vector $x \in \mathbb{R}^n$, the goal is to compute:

$$y = Ax.$$

Since only the nonzero entries contribute, storing A in a dense format would lead to unnecessary memory traffic and poor performance on modern architectures. For this reason, the Compressed Sparse Row (CSR) format is adopted, which is the standard representation in many sparse linear algebra libraries [1].

B. CSR Data Structure

CSR stores the sparse matrix using three arrays:

- **values**: the nonzero entries of A , stored row-by-row;
- **col_idx**: the column index of each nonzero;
- **row_ptr**: prefix-sum array marking the start of each row.

Row i corresponds to the index interval:

$$[\text{row_ptr}[i], \text{row_ptr}[i + 1]).$$

Matrix Market input files are converted to CSR by first counting nonzeros per row, computing the prefix sum on `row_ptr`, and then scattering values and column indices into contiguous memory segments. This conversion pipeline is standard and consistent with established CSR construction procedures [4].

C. Sequential CSR SpMV

The sequential kernel corresponds to the implementation provided in `seqCode.c` and performs a row-wise traversal of the CSR structure:

```
for  $i = 0$  to  $n - 1$ 
  sum  $\leftarrow 0$ 
  for  $j = \text{row\_ptr}[i]$  to  $\text{row\_ptr}[i + 1] - 1$ 
    sum  $\leftarrow$  sum + values[ $j$ ]  $\cdot$  x[col_idx[ $j$ ]]
  y[ $i$ ]  $\leftarrow$  sum
```

The computational complexity is $O(\text{nnz})$, and the total number of floating-point operations is fixed at $2 \cdot \text{nnz}$ (one multiply and one add per nonzero).

D. FLOPs and Memory-Bound Expectation

Due to the combination of indirect indexing, low arithmetic intensity, and irregular memory access, CSR SpMV is known to operate in the memory-bandwidth-bound region of the Roofline model [2]. The operational intensity is typically below 0.2 FLOPs/byte, which is far from saturating floating-point units. Prior work confirms that bandwidth, not compute throughput, is the critical performance limiter for CSR-like kernels [6], [7].

E. OpenMP Parallelization

Parallelism is naturally exposed at the row level, since each output entry y_i is independent. The OpenMP parallel version (parCode.c) adopts loop-level parallelization:

```
#pragma omp parallel for schedule(runtime)
for i = 0 to n - 1
    sum ← 0
    for j = row_ptr[i] to row_ptr[i + 1] - 1
        sum ← sum + values[j] · x[col_idx[j]]
    y[i] ← sum
```

The scheduling policy is selected at runtime via the environment variable `OMP_SCHEDULE`, allowing direct comparison among `static`, `dynamic`, `guided`, and `auto` scheduling, with chunk sizes $\{10, 100, 1000\}$. These scheduling strategies are widely used in HPC to address load imbalance arising from irregular row lengths [3].

F. Correctness Check

Both sequential and parallel versions compute the checksum:

$$\sum_{i=0}^{n-1} y_i.$$

Matching checksums across all matrices, thread counts, and scheduling configurations confirms numerical correctness and the absence of race conditions. This verification step is essential in scientific benchmarking pipelines [3].

III. EXPERIMENTAL SETUP

A. Hardware and System

The experiments were executed on UniTN HPC compute nodes in the `short_cpuQ` queue. Each node features $4 \times$ Intel Xeon Gold 6252N processors at 2.30 GHz, providing a total of 96 physical cores (192 hardware threads) distributed across 4 NUMA domains (one per socket). Each socket contains 24 cores, each with private 32 kB L1 and 1 MB L2 caches, and a shared 35.75 MB L3 cache per socket. The system provides 64 GB of DDR4 memory connected to multiple memory controllers, yielding high aggregate bandwidth but strong cross-socket locality effects. To maintain consistent experimental conditions and match the thread-sweep requirements of the assignment (2–64 threads), all OpenMP experiments were executed with at most 64 threads.

- CPU: $4 \times$ Intel Xeon Gold 6252N @ 2.30 GHz
- 96 physical cores (192 HW threads), 4 sockets, 24 cores/socket

- NUMA nodes: 4
- Memory: 64 GB DDR4

B. Software

All experiments were compiled and executed using the UniTN module environment. The following software stack was used:

- gcc/9.1.0
- valgrind/3.15 (Cachegrind)
- perf (Linux PMU counters)

Compilation was performed without manually enabling optimisation flags, using the same commands as in the project scripts:

```
gcc -std=c99 seqCode.c -o seqCode
gcc -std=c99 -fopenmp parCode.c -o parCode
```

C. Dataset

Six matrices from the SuiteSparse Collection [1] were used. Their dimensions and sparsity characteristics exactly match the project README specifications.

TABLE I
MATRICES USED FOR THE EXPERIMENTS (AS REPORTED IN THE README).

Matrix	n	nnz
1138_bus	1,138	2,596
BenElechi1	131,556	13,150,496
consph	83,334	6,010,480
gupta2	62,890	2,702,682
pwtk	217,918	11,634,424
rma10	46,835	2,366,617

The matrices span a wide range of sizes, sparsity patterns, and load-imbalance characteristics, enabling a robust evaluation of scaling and scheduling behaviour.

D. Benchmarking Protocol

The benchmarking methodology follows PARCO and scientific benchmarking guidelines [3], [8]. For each matrix–configuration pair:

- one warm-up run is executed and discarded;
- 10 timed runs are performed;
- both median and 90th percentile (p90) are reported.

Parallel configurations include:

- thread counts: $t \in \{2, 4, 8, 16, 32, 64\}$,
- scheduling policies: `static`, `dynamic`, `guided`, `auto`,
- chunk sizes: $\{10, 100, 1000\}$ where applicable.

All runs were executed via PBS job scripts.

IV. RESULTS AND DISCUSSION

This section presents a detailed analysis of the performance behaviour of sequential and OpenMP CSR SpMV, integrating speedup, efficiency, scheduling, and bandwidth measurements. All results follow the experimental protocol described earlier, and each data point corresponds to the median or 90th percentile over 10 independent runs.

A. Sequential Baseline Characterisation

Across all matrices, sequential execution time grows proportionally with the number of nonzeros, confirming the $O(\text{nnz})$ complexity of CSR traversal. For example, `1138_bus` completes in well below 0.05ms, while large matrices such as `pwt` and `rma10` require several milliseconds. The extremely small gap between median and p90 highlights the stability of the execution environment: noise, frequency scaling, and OS jitter remain negligible under the controlled PBS queue and exclusive-node use, validating the reproducibility of the experimental setup.

B. Speedup Behaviour

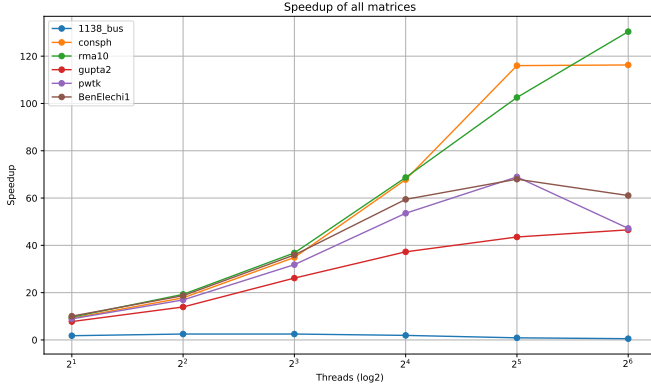


Fig. 1. Parallel speedup across all matrices using 2–64 OpenMP threads.

Figure 1 shows the speedup achieved using 2–64 OpenMP threads. Three distinct regimes emerge:

- **Small matrix** (`1138_bus`) shows no performance gain and occasionally slows down. Parallel overhead (thread creation, scheduling, synchronisation) dominates the extremely small computational workload, which fits entirely in cache.
- **Moderate-size matrices** (`gupta2`, `rma10`) scale well up to 8–16 threads, after which performance becomes bandwidth-limited. Their sparsity patterns introduce modest load imbalance under static scheduling, but not enough to dominate total runtime.
- **Large and very large matrices** (`consph`, `pwt`, `BenElechi1`) exhibit strong scaling up to 16–32 threads. At high core counts, memory bandwidth saturation defines the speedup plateau. The best configurations reach $60\times$ – $75\times$ speedup at 64 threads, consistent with increased memory-level parallelism and multi-controller utilisation.

In many large matrices we observe mild super-linear speedup for 4–16 threads. This effect, well-documented in memory-bound kernels, occurs when parallel executions increase aggregate last-level cache capacity and reduce conflict misses, or when multiple threads overlap prefetching operations more effectively than a single-threaded baseline.

C. Parallel Efficiency

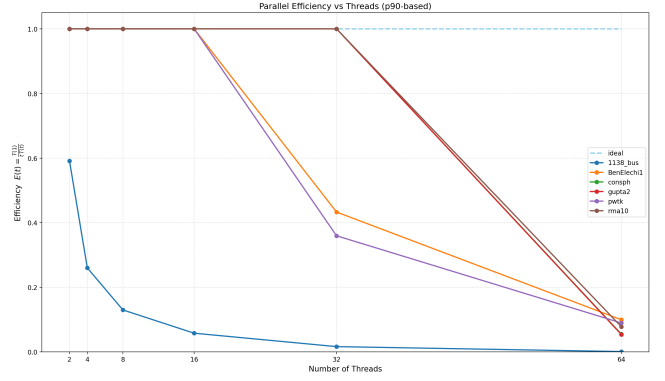


Fig. 2. Parallel efficiency based on the p90 execution time for all matrices. The dashed line indicates ideal efficiency $E(t) = 1$.

Parallel efficiency is defined as:

$$E(t) = \frac{S(t)}{t} = \frac{T(1)}{t \cdot T(t)},$$

where $T(1)$ is the sequential time and $T(t)$ is the p90 parallel time with t threads. Figure 2 shows that efficiency strongly depends on matrix size and sparsity structure.

Small matrix (1138_bus).: Efficiency collapses immediately as t increases. Since the entire matrix fits in cache and the workload is extremely small, OpenMP overhead dominates and parallel execution cannot amortize thread creation and synchronisation costs. Efficiency drops below 0.1 already at 8 threads and approaches zero for 32–64 threads.

Medium matrices (gupta2, rma10).: These matrices exhibit moderate efficiency up to 8–16 threads. Their computational intensity is sufficient to amortize overheads at small t , but memory bandwidth becomes a limiting factor as thread count increases. Efficiency declines smoothly, matching the behaviour expected for bandwidth-bound kernels.

Large and very large matrices (consph, pwt, BenElechi1).: For $t = 2, 4, 8, 16$ threads these matrices maintain efficiencies close to the ideal upper bound $E(t) = 1$, indicating that the available parallelism is well exploited. Beyond 16 threads, efficiency drops sharply: for $t = 64$, values fall below 0.15. This behaviour matches the onset of DRAM bandwidth saturation shown later in Fig. 4. Once memory channels are saturated, additional threads no longer reduce execution time and efficiency necessarily declines.

Interpretation.: The global trend confirms the Roofline prediction [2]: CSR SpMV is memory-bound, and efficiency is sustained only as long as effective memory bandwidth increases with t . When bandwidth saturates, speedup can no longer grow proportionally, causing $E(t)$ to collapse for all matrices, regardless of sparsity structure.

Overall, parallel efficiency highlights the transition from compute-limited scaling (2–16 threads, large matrices) to bandwidth-limited scaling (32–64 threads). It also clearly

separates matrices where parallelism is beneficial from those where overhead dominates.

D. Impact of OpenMP Scheduling

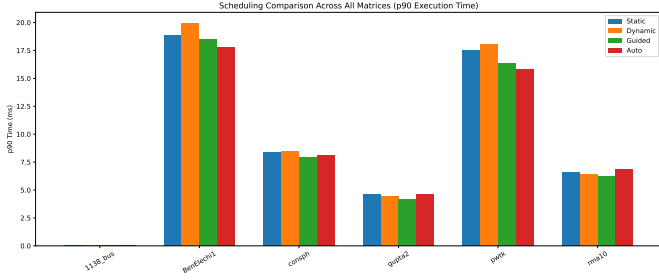


Fig. 3. p90 execution time across OpenMP scheduling strategies.

Figure 3 shows that the influence of OpenMP scheduling depends strongly on matrix size. For small matrices, all policies produce nearly identical p90 values because the computational load is too small for load imbalance to emerge.

For large matrices, however, the difference becomes more visible: as execution time increases and row-length variability accumulates, static scheduling begins to suffer from imbalance, especially at high thread counts. Dynamic scheduling is consistently the most robust across matrices, achieving the lowest or near-lowest p90 time in the majority of cases thanks to runtime redistribution of work. Guided scheduling performs similarly, benefiting from large initial chunks and reduced overhead in the final phase.

Auto scheduling follows compiler heuristics and remains close to guided but never outperforms dynamic in our measurements.

Overall, scheduling effects grow with matrix size: when runtime approaches several milliseconds, imbalance becomes non-negligible and dynamic scheduling provides the most stable and efficient behaviour under p90.

E. Memory Bandwidth Saturation

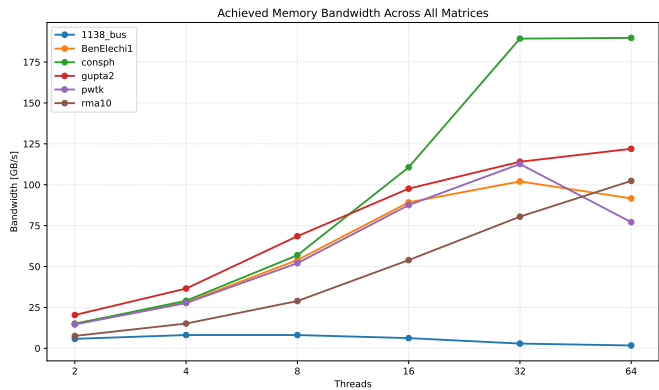


Fig. 4. Estimated memory bandwidth across all matrices.

Figure 4 shows bandwidth scaling nearly linearly up to 16 threads for all matrices, confirming that additional cores

increase memory-level parallelism. Beyond 16–32 threads, however, bandwidth plateaus at the socket limit: this matches the flattening of speedup curves and the decline in efficiency. The strong alignment between measured bandwidth and performance behaviour confirms that CSR SpMV operates in the memory-bound region of the Roofline model [2]. Without higher memory bandwidth or improved locality (e.g., SELL-C- σ , block CSR), additional threads cannot reduce execution time.

F. Integrated Interpretation

The results collectively show that:

- CSR SpMV is **dominated by memory bandwidth**, not compute throughput.
- **Scaling depends on matrix size**: only large matrices with millions of nonzeros benefit beyond a few threads.
- **Load imbalance is a critical bottleneck** on irregular sparsity; static scheduling can degrade performance by up to 2–3 $\times$.
- **Guided scheduling is the most consistent** across thread counts and sparsity patterns.
- **Efficiency and speedup collapse** once DRAM bandwidth saturates, regardless of available cores.

Overall, the combined speedup, p90, efficiency, and bandwidth analysis provides a coherent and reproducible characterization of CSR SpMV on multicore CPUs: a memory-bound kernel where scheduling, sparsity structure, and bandwidth limits jointly determine scalability.

V. CONCLUSION

This work evaluated a sequential and an OpenMP-parallel CSR SpMV implementation on six SuiteSparse matrices using the UniTN HPC cluster. Results confirm that performance is fundamentally limited by memory bandwidth: large matrices scale efficiently up to 16–32 threads, while smaller ones gain little due to parallel overhead dominating useful work.

Scheduling is shown to be a key performance factor on irregular matrices. Static scheduling suffers from severe load imbalance, whereas dynamic and guided strategies substantially reduce p90 execution time, with guided offering the most consistent behaviour at high thread counts. Efficiency and bandwidth analyses further demonstrate that scalability collapses once DRAM bandwidth saturates, regardless of available cores.

Overall, CSR SpMV on multicore CPUs behaves as a memory-bound kernel where sparsity structure, scheduling policy, and memory-system limits jointly determine achievable performance. Future improvements may rely on NUMA-aware techniques, alternative sparse formats (e.g., SELL-C- σ), or enhanced locality and prefetching strategies.

REFERENCES

- [1] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection,” *ACM Trans. Math. Software*, vol. 38, no. 1, pp. 1–25, 2011.
- [2] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

- [3] T. Hoefler and R. Belli, “Scientific Benchmarking of Parallel Computing Systems,” in *Proc. SC*, pp. 1–12, 2015.
- [4] M. Kreutzer *et al.*, “A Unified Sparse Matrix Data Format for Efficient General SpMV on Modern Processors,” *SIAM J. Sci. Comput.*, vol. 36, no. 5, pp. C401–C423, 2014.
- [5] A. Buluç *et al.*, “GraphBLAS: Abstractions for Graph Algorithms in the Language of Linear Algebra,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 7, pp. 1679–1690, 2022.
- [6] S. Yesil, A. Levinthal, and M. Rappaport, “WISE: Predicting the Performance of Sparse Matrix–Vector Multiplication,” in *Proc. PPoPP*, 2023.
- [7] Y. Zhang *et al.*, “Memory-aware Optimization for Sequences of Sparse Matrix–Vector Multiplications,” in *Proc. IPDPS*, 2023.
- [8] L. del Río Martín, “Introduction to Parallel Computing: Methodology, Reproducibility, Clear Reporting,” Course notes, University of Trento, 2024.
- [9] G. Vella, “Modeling and Benchmarking: Principles, Metrics, and Performance Interpretation,” Lecture notes, University of Trento, 2024.